

Week 5 — Lab Guide (2-Day)

Prompting + Eval Literacy + M1 [DR #1]

Phase 1 · Foundations, Eval Literacy & System Design · CLOSING WEEK

What you'll have built by Friday

The W4 contract — `/ask` , `/ask_batched` , `/health` , the structured `Answer` — is unchanged. What's new is everything around measurement:

- A **Stakeholder Map** committed to `docs/adr/0001-stakeholder-map.md` that formally locks the use case for M1.
- A **20-entry golden set** in `data/golden_set.jsonl` , mixed happy-path / harder / edge — the ground truth every later week's KPI is measured against.
- An **LLM-as-judge** in `src/eval/judge.py` that scores any candidate answer against an ideal answer on three rubric dimensions, persisting to the new `eval_runs` table in SQLite.
- A **pairwise comparison** between two prompt variants, with random position-flip to mitigate position bias.
- A **Critic-Creator trace** showing systematic prompt improvement over 2–3 rounds on one hard question.
- An **ADR v1 (Locked)** that consolidates the W1 framing + W4 cost budget
 - W5 eval baseline, ready to defend at DR #1.
- A **DR #1 1-pager** in `docs/dr/dr1-summary.md` — what you built, what you measured, what you want to discuss.

Time budget: ~3.5 hours of lab work across two days, plus the ~15-minute DR #1 slot scheduled separately later in the week.

API spend: ~\$1.00 per learner — the `gpt-4o` judge dominates the cost.

Bundle reference

Everything you need is in the W5 bundle. The canonical path on the left is where the file lives in your cohort repo; the bundle filename on the right

is what you downloaded.

Canonical path (cohort repo)	Bundle file
src/eval/judge.py	AI-RAG_W5_judge_starter.py / _reference.py
src/eval/pairwise.py	AI-RAG_W5_pairwise_reference.py
src/eval/critic_creator.py	AI-RAG_W5_critic_creator_reference.py
src/eval/golden.py	AI-RAG_W5_golden_reference.py
src/pipeline/store.py	AI-RAG_W5_store_reference.py (extends W4)
scripts/run_eval.py	AI-RAG_W5_run_eval.py
scripts/run_pairwise.py	AI-RAG_W5_run_pairwise.py
scripts/run_critic_creator.py	AI-RAG_W5_run_critic_creator.py
data/golden_set.jsonl	AI-RAG_W5_golden_set_sample.jsonl (your 3-entry start)
data/golden_set_full.jsonl	AI-RAG_W5_golden_set_full.jsonl (20-entry reference)
docs/stakeholder-map-template.md	AI-RAG_W5_stakeholder_map_template.md
docs/golden-set-notes.md	AI-RAG_W5_golden_set_notes_template.md
docs/eval-run-001.md	AI-RAG_W5_eval_run_001_template.md
docs/prompt-pairwise-001.md	AI-RAG_W5_prompt_pairwise_001_template.md
docs/critic-creator-trace.md	AI-RAG_W5_critic_creator_trace_template.md
docs/adr/0001-capstone-framing.md	AI-RAG_W5_adr_0001_capstone_framing_v1_locked.md
docs/design-review-rubric.md	AI-RAG_W5_design_review_rubric.md
docs/dr/dr1-summary.md	AI-RAG_W5_dr1_summary_template.md

docs/dr/dr1-peer-review-
form.md

AI-RAG_W5_dr1_peer_review_form.md

***Starter vs reference files.** The judge file has both — start with the starter, fill the three TODOs in Step 3 (“3a write rubric”, “3b call the client”, “3c parse the tool_call”). The reference shows the worked shape; use it to check your work or copy the rubric prompt’s structure if you’re stuck. Other files are drop-in references — you read them, you don’t rewrite them.*

Pre-flight

Before you start: open Day 1 of the slide deck on a second monitor. The **Lab Step 1+2 callout** on slide 17 and **Lab Steps 3-5 callout** on slide 30 mirror this guide. The deck has the *why*; this guide has the *how*.

Also have these open:

- The **Design Review Rubric** (docs/design-review-rubric.md) — pre-read for DR #1. Skim the 5×4 table once before Lab Step 6.
 - The **DR #1 1-pager template** — peek at the structure now so the M1 push on Day 2 doesn’t surprise you.
-

Step 0 — Environment (15 min)

0a — Confirm W4 is green before you change anything

What and why

You’ll extend the SQLite store with an eval_runs table this session, build a new src/eval/ module, and call the cohort’s /ask_batched endpoint from a script. If W4 is broken, all of that breaks. Confirm W4 first.

Where

Your cohort repo root.

About to change

Nothing — this is a sanity check.

Make the change

Nothing to edit. We're just running the W4 tests and the W4 API.

Run it

```
# Activate your venv
source .venv/bin/activate

# W4 tests must be green before W5 begins
pytest tests/ -q

# Start the W4 API in another terminal (or backgrounded)
uvicorn src.api.main:app --reload --port 8000 &
sleep 2

# Confirm /ask_batched returns the W4 shape (6 fields including schema_version)
curl -s -X POST http://localhost:8000/ask_batched \
  -H 'Content-Type: application/json' \
  -d '{"question": "What is RAG?"}' | python -m json.tool
```

See

```
..... [100%]
19 passed in 0.53s

{
  "content": "...",
  "cost_usd": 0.000064,
  "retries": 0,
  "confidence": 0.9,
  "sources": [],
```

```
"schema_version": "v1"  
}
```

What happened

W4 is intact — `Answer` has six fields, `cost_usd` is real, tests pass. You're ready to extend.

Stop uvicorn (`kill %1` or Ctrl-C in its terminal). You'll restart it when Lab Step 3 needs it.

0b — Extend the SQLite store with the `eval_runs` table

What and why

Lab Steps 3–5 each persist results to SQLite. They all use a new table — `eval_runs` — that records one row per (golden_id × judge run) tuple. Adding the table now means every later sub-step “just works”.

Where

`src/pipeline/store.py` — extends the W4 shape.

About to change

Replace the W4 `store.py` with the W5 reference. The new file adds an `eval_runs` table via the same idempotent `ensure_schema()` pattern from W4 (so existing `data/answers.db` files are safely upgraded).

Make the change

Copy the W5 reference into place:

```
cp <bundle>/AI-RAG_W5_store_reference.py src/pipeline/store.py
```

The W4 `answers` table is unchanged. The new `eval_runs` table has columns: `id`, `golden_id`, `question`, `candidate_answer`, `ideal_answer`, `candidate_model`, `judge_model`, `accuracy`, `groundedness`, `format`, `reasoning`, `eval_run_label`, `created_at`.

Run it

```
# Trigger the migration by calling ensure_schema once
python -c "from src.pipeline.store import ensure_schema; ensure_schema('data/answers.db', 'eval_runs')"

# Verify the new table exists
sqlite3 data/answers.db ".schema eval_runs"
```

See

```
CREATE TABLE eval_runs (
  id          INTEGER PRIMARY KEY AUTOINCREMENT,
  golden_id   TEXT NOT NULL,
  question    TEXT NOT NULL,
  candidate_answer TEXT NOT NULL,
  ideal_answer TEXT NOT NULL,
  candidate_model TEXT NOT NULL,
  judge_model TEXT NOT NULL,
  accuracy    INTEGER NOT NULL,
  groundedness INTEGER NOT NULL,
  format      INTEGER NOT NULL,
  reasoning   TEXT NOT NULL,
  eval_run_label TEXT DEFAULT 'eval-run-001',
  created_at  TEXT DEFAULT CURRENT_TIMESTAMP
);
```

If you rerun `ensure_schema`, it's idempotent — the table is left alone. That's the same discipline as the W4 migration; safe to run anytime.

What happened

Your database is W5-ready. The `answers` rows from W2/W3/W4 are untouched; the new `eval_runs` table is ready to receive rows from Lab Steps 3-5.

Step 1 — Stakeholder Mapping

Worksheet (20 min)

This step **locks the use case for M1**. The W5 lesson plan moved this out of the live session (originally a 30-min in-class exercise in old W5) into a self-paced lab worksheet — both for time reasons and because the worksheet works better with thinking time than with a room.

1a — Open the worksheet template

What and why

The worksheet has 5 prompts. Filling all 5 takes ~20 minutes if you've done the prep (you know your primary user and your sponsor). Less time than that probably means you're hand-waving on at least one prompt.

Where

Your cohort repo, `docs/` folder.

About to change

Copy the template into place.

Make the change

```
cp <bundle>/AI-RAG_W5_stakeholder_map_template.md \
  docs/stakeholder-map-template.md
```

Open the file in your editor. Scan the five prompts:

1. Primary user — who exactly?
2. Secondary stakeholders — who else has skin in this?
3. Before-AI workflow — how is this solved today?
4. Desired after-state — what changes?
5. Top 3 sponsor metrics — what does the sponsor measure?

Run it

```
wc -l docs/stakeholder-map-template.md
```

See

```
~90 docs/stakeholder-map-template.md
```

What happened

The template is in your repo. You're not editing the template directly — in 1b you'll fill it in and save it as the locked artefact.

1b — Fill the five prompts honestly

What and why

The point of this worksheet is **specificity**. Vague stakeholder maps fail DR #1. *"Employees"* is not a primary user; *"engineering managers running 1:1s with reports who hit policy questions"* is. Push yourself.

Where

`docs/stakeholder-map-template.md` (filling) `docs/adr/0001-stakeholder-map.md` (committed).

About to change

You'll write ~400-500 words across the five prompts. Most of it is the *Primary user* and *Top 3 metrics* sections; the others are typically shorter.

Make the change

Fill the template in your editor. Use the examples in the worksheet as anchors, NOT as templates to fill in — your domain is different.

A few quality checks while you write:

- **Primary user.** Name a *role + context*, not just a role. Bad: “HR team.” Good: “HR business partners during quarterly performance review cycles.”
- **Secondary stakeholders.** Include at least one *gatekeeper* (Security, Compliance, Legal) and one *whose-work-is-augmented* role — these are the stakeholders most often forgotten.
- **Before-AI workflow.** Be concrete about time. “*Takes a while*” is not a number; “*~5 minutes per question, ~3 questions per 1:1, ~6 hours/manager/month*” is.
- **Sponsor metrics.** All three must be *measurable*. “*User satisfaction*” without a measurement plan is a vibe, not a metric.

When the worksheet is filled, save as `docs/adr/0001-stakeholder-map.md` (note the path — it lives under `adr/` because it’s part of the formal-decision-record stack).

Run it

```
mkdir -p docs/adr
cp docs/stakeholder-map-template.md docs/adr/0001-stakeholder-map.md
# Now edit the new file — fill the prompts.
$EDITOR docs/adr/0001-stakeholder-map.md
```

See

After saving:

```
grep -c "^>" docs/adr/0001-stakeholder-map.md
```

should show many fewer `>` blockquote lines than the template (you’ve replaced the example blockquotes with your own paragraphs). The template has ~15 blockquote lines; a filled-in version typically has 0–5.

What happened

Your use case is captured in writing. From here on, every decision in the capstone — model, cost budget, KPIs, retrieval strategy — gets defended against *this* user, not a generic “users”.

1c — Commit and lock

What and why

Once committed, the file is the M1 deliverable for “locked use case”. The locking is what makes it usable as ground truth across the rest of the programme — DR #2, DR #3, every later week’s KPI conversation references it.

Where

Repo root.

About to change

Git commit.

Make the change

Stage and commit:

```
git add docs/stakeholder-map-template.md docs/adr/0001-stakeholder-map.md
git commit -m "M1 prep — stakeholder map locked: <one-line capstone summary>"
```

Run it

```
git log --oneline -1
```

See

```
<sha> M1 prep — stakeholder map locked: HR-policy assistant for engineering manage
```

What happened

M1 deliverable (d) — Stakeholder Map — is committed. Move on to Step 2.

Step 2 — Build the 20-Sample Golden Set

(45 min)

The 20-entry golden set is **the most important deliverable of the week**.

Every later week's KPI baseline is computed against these 20 questions.

The W5 lesson plan specifies **14 happy-path + 4 harder + 2 edges**.

Resist the temptation to write all 20 quickly — quality beats quantity.

Most learners get the best mileage from spending the full 45 minutes.

2a — Sketch the categories on paper

What and why

Before opening jsonl, sketch on paper or a scratch buffer: what are the 14 happy-path categories your capstone needs to cover? What 4 harder questions interleave them? What 2 edges?

Where

Anywhere — scratch markdown, a notebook, paper.

About to change

Nothing in the repo yet.

Make the change

Sketch a quick table:

Happy-path (14):

1. <category — e.g. leave> — q1, q2
2. <category — e.g. expenses> — q3, q4
3. <category — e.g. working-week> — q5, q6
- ...

Harder (4):

15. <multi-section — e.g. remote × meetings>

16. <conditional — e.g. parental × incident>

...

Edges (2):

19. <not in docs — e.g. crypto investing>

20. <out of scope — e.g. personal-trip visa>

The point is to spread your 14 happy-path entries across **multiple policy areas** — not 14 leave-policy questions.

Run it

N/A — sketching.

See

A table that maps 20 ids to 20 specific question seeds. ~5 minutes work.

What happened

You have a coverage plan. Now you write entries that fill the plan, instead of writing entries and hoping the coverage works out.

2b — Write the 14 happy-path entries

What and why

Happy-path entries should be **real user-style questions**. Best sourcing: real questions, support tickets, FAQ lists. Second best: a domain expert imagining a user. Worst: you, alone, making up questions.

Where

`data/golden_set.jsonl` — append one JSON object per line.

About to change

Start from the 3-entry sample provided in the bundle. You'll grow it to 14 entries in this sub-step (covering happy-path categories from 2a).

Make the change

Copy the sample into place:

```
cp <bundle>/AI-RAG_W5_golden_set_sample.jsonl data/golden_set.jsonl
```

Then open `data/golden_set.jsonl` in your editor. The 3 sample entries are EKA-flavoured — adapt or replace them for your own capstone domain. Each entry is one line:

```
{"id": "g001", "question": "...", "ideal_answer": "...", "notes": "happy-path. Source: ..."} 
```

For each of your 14 happy-path entries:

- Pick an `id` from `g001` to `g014` .
- Write the `question` as a real user would phrase it (not as you'd formally describe it).
- Write the `ideal_answer` as a real, full answer. This is what the judge uses as reference. If your domain answers can vary (e.g. multiple valid phrasings), put the *facts* in the ideal answer, not one phrasing.
- Write `notes` starting with `happy-path.` — the bucket prefix is read by `validate_golden_set()` to compute coverage.

For inspiration, peek at `AI-RAG_W5_golden_set_full.jsonl` — it's the 20-entry EKA reference. **Don't copy entries directly** — your capstone isn't an HR assistant unless it is.

Run it

```
wc -l data/golden_set.jsonl  
python -c "from src.eval.golden import validate_golden_set; r = validate_golden_set('data/golden_set.jsonl')"
```

See

```
14 data/golden_set.jsonl  
{'happy': 14}
```

What happened

14 happy-path entries committed. The validator confirms the coverage bucket. Now the harder ones.

2c — Write the 4 harder entries

What and why

Harder entries test **multi-hop reasoning** or **integration across two source sections**. They're where the eval results often surprise you — the model gets the easy stuff but slips on the multi-hop.

Where

`data/golden_set.jsonl` — append 4 more.

About to change

Add entries `g015` through `g018`.

Make the change

Write 4 harder entries. Examples of harder-question shapes:

- **Multi-section.** “How does policy X interact with policy Y?” — the answer requires reading two sections.
- **Conditional.** “If I’m in situation A, but also have constraint B, what’s the right path?” — the answer depends on combining rules.
- **Process across teams.** “When I need to do X involving teams A and B, what’s the right sequence?” — requires understanding multiple stakeholders’ processes.
- **Edge of policy clarity.** “The policy says X but my situation is Y — does that apply?” — requires reasoning about scope, not just retrieving the rule.

In `notes`, start with `harder.` (or `harder · multi-hop.` — the validator buckets on the first word).

The 4 harder entries in `golden_set_full.jsonl` show the shape: g015 (remote-work × meeting requirement), g016 (parental leave × incident

response), g017 (conflict of interest disclosure), g018 (Coursera out-of-catalogue with conditions).

Run it

```
wc -l data/golden_set.jsonl  
python -c "from src.eval.golden import validate_golden_set; r = validate_golden_set('data/
```

See

```
18 data/golden_set.jsonl  
{'happy': 14, 'harder': 4}
```

What happened

Coverage extends to harder. Two edges left.

2d — Write the 2 edge entries

What and why

Edge entries test **graceful refusal**. They're questions where the correct answer is "I don't know" or "that's outside my scope". A naive capstone hallucinates here — invents a plausible answer for something that isn't in its docs. The eval surfaces this immediately.

Where

data/golden_set.jsonl — append the last 2.

About to change

Add entries g019 and g020 .

Make the change

Write 2 edge entries. Two shapes that work:

- **Topic genuinely not in your docs.** A question your capstone *can't* answer because the corpus doesn't cover it. The ideal answer says so clearly and suggests where the user should go instead.
- **Topic adjacent to your docs but out of scope.** A question that *sounds* like it might be in scope but isn't. The ideal answer redirects without inventing.

In `notes` , start with `edge.` so the validator buckets correctly.

The 2 edge entries in `golden_set_full.jsonl` :

- g019 (cryptocurrency investing — genuinely not in HR policies)
- g020 (personal-trip visa — adjacent to travel policy but not its scope)

Run it

```
python -c "from src.eval.golden import validate_golden_set; r = validate_golden_set('data/
```

See

```
{'path': 'data/golden_set.jsonl', 'n': 20, 'ids': ['g001', ..., 'g020'], 'coverage': {'happy': 14, 'ha
```

What happened

Coverage is the lesson-plan ideal: 14/4/2. The golden set is ready to be used by the judge in Step 3.

2e — Write the golden-set notes + commit

What and why

The notes document explains *how you chose* these 20 entries — sourcing, bucket reasoning, what was hard. It's the part DR #1 reviewers read when they want to know whether your golden set is real or made-up.

Where

`docs/golden-set-notes.md` .

About to change

Copy the template, fill it in.

Make the change

```
cp <bundle>/AI-RAG_W5_golden_set_notes_template.md docs/golden-set-notes.md
$EDITOR docs/golden-set-notes.md
```

Fill three sections:

- **Coverage mix.** Confirm 14/4/2 (it should match the validator output).
- **Sourcing.** Be honest. Where did each bucket come from? Real users is best; the more “made up by me” entries you have, the weaker your baseline.
- **Hardest to write.** A short paragraph naming which bucket was hardest. Most learners say the edge cases.

Then commit everything:

```
git add data/golden_set.jsonl docs/golden-set-notes.md
git commit -m "M1 prep — golden_set.jsonl with 20 entries + coverage notes"
```

Run it

```
git log --oneline -2
```

See

```
<sha2> M1 prep — golden_set.jsonl with 20 entries + coverage notes
<sha1> M1 prep — stakeholder map locked: ...
```

What happened

M1 deliverable (e) — the golden set + notes — is committed. Day 1 lab work done. Step 3 is tomorrow (Day 2).

Step 3 — Run LLM-as-Judge on the Golden Set (45 min)

This is where the eval mental shift becomes operational. You build (or verify) `judge.py`, point it at your 20 golden entries, and produce the **M1 evaluation baseline** that every later week measures against.

3a — Build (or verify) `judge.py`

What and why

The starter has three TODOs from the live session:

- **3a — write `RUBRIC_PROMPT`** . The 200-word prompt that defines the three dimensions for your domain. The single most-leveraged 200 words of the week.
- **3b — call `client.chat.completions.create`** with the tool-calling pattern (same shape as W4).
- **3c — parse the `tool_call` into a `JudgeScore`** .

If you wrote `judge.py` in the live session at slide 25's demo, this sub-step is just confirming. Otherwise, fill the TODOs now.

Where

`src/eval/judge.py` .

About to change

Replace the `RUBRIC_PROMPT = "TODO..."` placeholder with your full rubric prompt. Replace `resp = None` with the OpenAI client call. Replace the `raise NotImplementedError` with the json-parse + `JudgeScore` construction.

Make the change

Copy the starter:

```
cp <bundle>/AI-RAG_W5_judge_starter.py src/eval/judge.py
```

Then fill the three TODOs. The reference at

`AI-RAG_W5_judge_reference.py` shows a complete EKA-flavoured version —

adapt the dimension definitions to your own capstone domain. The

levels (1 / 2 / 3 / 4 anchors) are what you customise; the structure

stays put.

Three things to get right in your rubric:

- **Frame the judge.** One sentence: “You are a senior evaluator for a ”.
- **Anchor each level.** Don’t leave 1 / 2 / 3 / 4 as adjectives. Write one concrete sentence per level per dimension. The reference shows the shape.
- **Force tool use.** End with: “ALWAYS call the rate_answer tool. Never reply in plain text.”

The full prompt should be **at least 150 words**. Short rubrics produce noisy scores — see the W5 deck slide 24 mitigations.

Run it

```
# Confirm imports and the rubric prompt landed (length check)
python -c "
from src.eval.judge import RUBRIC_PROMPT
words = len(RUBRIC_PROMPT.split())
print(f'rubric prompt: {words} words')
assert words > 100, 'rubric is too short — see reference'
"

# Confirm unit tests pass — these check the schema + parsing logic
pytest tests/test_judge.py -q
```

See

```
rubric prompt: 280 words
..... [100%]
13 passed in 0.18s
```

What happened

`judge.py` is complete and tested. Every later step in W5 calls `judge_one()` — and so do W6, W11, W17, every later week's eval.

3b — Run the judge on all 20 golden entries

What and why

The script (`scripts/run_eval.py`) loads the golden set, calls your W4 `/ask_batched` to get candidate answers, calls `judge_one` to score them, and persists each verdict to the `eval_runs` table. About \$0.20–0.40 of OpenAI credits for one run.

Where

Two terminals — `uvicorn` + the eval script.

About to change

Nothing in code. You're running.

Make the change

Copy the script if you haven't:

```
cp <bundle>/AI-RAG_W5_run_eval.py scripts/run_eval.py
```

Make sure `OPENAI_API_KEY` is exported and the W4 API is up.

Run it

```
# Terminal 1
uvicorn src.api.main:app --reload --port 8000 &
sleep 2

# Terminal 2
python scripts/run_eval.py \
  --golden-set data/golden_set.jsonl \
  --db      data/answers.db \
  --api-url http://localhost:8000 \
```

```
--judge-model gpt-4o \
--label    eval-run-001
```

See

About 60-90 seconds of progress logs as it works through the 20 entries, then:

```
=====
Aggregate for label='eval-run-001'
-----
n entries scored   : 20
avg accuracy      : 3.45
avg groundedness  : 3.20
avg format        : 3.65
accuracy 4 / 3 / 2 / 1 : 11 / 7 / 2 / 0
wall-clock       : 87.4s
```

Write a 1-page summary in docs/eval-run-001.md — the template is in the bundle. This is your M1 eval baseline.

Your exact numbers will differ. **Numbers in the 3.0–3.6 range across all three dimensions are typical** for a W4 `gpt-4o-mini` candidate against a strong-judge rubric. If your numbers are all 4.0, your rubric is too lenient — sharpen the Level-3 anchors. If your numbers are below 2.5, either your `/ask_batched` is broken or your golden set's ideal answers need work.

What happened

20 rows in the `eval_runs` table, labelled `eval-run-001`. The first real measurement of your capstone's quality. Now we read it.

3c — Spot-check the judge by reading 5 entries

What and why

The W5 deck's mitigation #3 is **spot-check 10%** of judge verdicts yourself. With 20 entries, that's 2 — but read 5 for the first run.

This is the discipline that catches a too-lenient rubric or a genuinely-wrong candidate.

Where

SQLite — read the rows.

About to change

Nothing in code or data. You're inspecting.

Make the change

Nothing to edit.

Run it

```
# Pull 5 rows including the judge's reasoning
sqlite3 -header -column data/answers.db <<'SQL'
SELECT
  golden_id AS id,
  accuracy AS a,
  groundedness AS g,
  format AS f,
  substr(candidate_answer, 1, 80) AS candidate_snip,
  substr(reasoning, 1, 100) AS judge_reasoning
FROM eval_runs
WHERE eval_run_label = 'eval-run-001'
ORDER BY RANDOM()
LIMIT 5;
SQL
```

See

Five random rows, each with the candidate answer's first 80 chars and the judge's reasoning's first 100 chars. Read each one. Ask yourself:

- Do *you* agree with the score? If the judge said `accuracy=4` but you can see a hedge or a missing detail, the judge is being lenient.

- Does the reasoning make sense? Did the judge cite specific things in the candidate, or did it produce a generic “the answer is good”?
- Are any candidates failing in a way the judge missed? Hallucinations (claims not in the ideal) are the most-missed failure mode.

What happened

You have a manual sanity check on the automated judge. If you found 1+ disagreements, that’s normal — the spot-check is the mitigation working. If you found 3+ on 5 rows, your rubric needs sharpening before the next eval run.

3d — Write `eval-run-001.md`

What and why

The aggregate stats and your spot-check observations together are the M1 eval baseline. The document is what reviewers read at DR #1.

Where

`docs/eval-run-001.md` .

About to change

Copy the template, fill it in.

Make the change

```
cp <bundle>/AI-RAG_W5_eval_run_001_template.md docs/eval-run-001.md
$EDITOR docs/eval-run-001.md
```

Fill in:

- **Aggregate stats** — paste from the script output above.
- **Per-entry breakdown** — pull from SQLite (the template has the SQL).
- **Three things you found** — the most important section:
 - Where you’re strong (which categories scored 4/4)
 - Where you’re weak (which scored ≤ 2 on at least one dimension)
 - Where you disagreed with the judge (from the spot-check)

- **What you'll change before DR #1** — at most 3 concrete actions.

Run it

```
git add scripts/run_eval.py src/eval/judge.py docs/eval-run-001.md
git commit -m "M1 prep — eval-run-001 baseline (avg accuracy <your number>)"
```

See

```
<sha> M1 prep — eval-run-001 baseline (avg accuracy 3.45)
```

What happened

M1 deliverable (e) — the eval baseline — is committed. This is the single most-scrutinised artefact at DR #1. From here on, every model change, prompt change, and retrieval change you make should re-run this script and update the trend.

Step 4 — Pairwise Comparison Between Two Prompts (30 min)

You've measured the current capstone against a rubric. Now you compare **two prompt variants** to learn which one is better. This is the pattern you'll use for every prompt change from W6 onwards.

4a — Write a v2 prompt with one specific change

What and why

The pairwise test only works if you change **one thing** between v1 and v2. Multiple changes mean you can't attribute the win/loss to any single factor. Pick one specific change.

Where

Two text files — `data/prompt-v1.txt` and `data/prompt-v2.txt` .

About to change

Write both files. v1 is your current `/ask` system prompt (the one from W3 or W4); v2 is v1 + one specific change.

Make the change

Save v1:

```
# Pull your current system prompt out of src/pipeline/pipeline.py or wherever
# you keep it. Save as v1.
cat > data/prompt-v1.txt << 'EOF'
You are an Enterprise Knowledge Assistant. Answer the user's question
using only the retrieved context. If the answer isn't in the context,
say so clearly.
EOF
```

Then write v2 with one specific change. Examples of single-change v2 variants:

- v2 adds a citation requirement: *"Cite the source section number when available."*
- v2 adds an explicit refusal instruction: *"If you're not certain, say 'I don't know' rather than guessing."*
- v2 adds a length constraint: *"Keep the answer under 80 words unless the question explicitly asks for more detail."*
- v2 adds a structure: *"Format: one short answer sentence, then a one-paragraph elaboration if needed."*

Write the v2 file:

```
cat > data/prompt-v2.txt << 'EOF'
You are an Enterprise Knowledge Assistant. Answer the user's question
using only the retrieved context. If the answer isn't in the context,
say so clearly.

Cite the source section number (e.g. "HR-policy.pdf §3.1") when the
answer comes from a specific policy document. If multiple sections are
involved, list each. If no citation is available, say "(no citation
```

```
available)".  
EOF
```

Run it

```
ls data/prompt-*.txt  
wc -w data/prompt-*.txt
```

See

```
data/prompt-v1.txt  
data/prompt-v2.txt  
  30 data/prompt-v1.txt  
  66 data/prompt-v2.txt  
  96 total
```

What happened

Your two prompt variants are committed. **The single change is named in your head** — when you write up the result in 4c, you'll cite it.

4b — Run the pairwise comparison

What and why

The script calls `/ask_batched` twice per golden entry (once with v1's system prompt, once with v2's), then calls `pairwise_compare` to judge which is better. Position-flip mitigates the bias. About \$0.40–0.80 of OpenAI credits.

Where

Same setup as Step 3.

About to change

Nothing in code. Running.

Make the change

Copy the script if you haven't, and confirm the W4 API is up:

```
cp <bundle>/AI-RAG_W5_run_pairwise.py scripts/run_pairwise.py
```

```
# Confirm uvicorn from Step 3 is still running, or restart:
```

```
# uvicorn src.api.main:app --reload --port 8000 &
```

***One catch:** the script assumes your `/ask_batched` accepts a `system_prompt` field in the request body. If your W3/W4 endpoint doesn't (most don't out of the box), the simplest workaround is to set `Settings.system_prompt` from each prompt file before launching uvicorn — run the script once per prompt, kill uvicorn between runs. See the script docstring for both flows.*

Run it

```
python scripts/run_pairwise.py \  
  --prompt-v1 data/prompt-v1.txt \  
  --prompt-v2 data/prompt-v2.txt \  
  --golden-set data/golden_set.jsonl \  
  --api-url http://localhost:8000 \  
  --judge-model gpt-4o \  
  --output data/pairwise-001-results.json
```

See

About 2-3 minutes of progress logs, then:

```
=====
Pairwise summary
-----
v1 wins    : 6
v2 wins    : 11
tie        : 2
ambiguous  : 1 (forward/reverse disagreed — position bias on these)
position-bias rate : 1/20 = 5%
wall-clock : 142.3s
```

Per-question results saved to data/pairwise-001-results.json

Your numbers will differ. **A position-bias rate under 15% is healthy.** Higher than 25% means your judge has issues on these question types — investigate the ambiguous rows before trusting the aggregate.

What happened

You have a clear-ish answer: v2 won more than v1 (or vice versa, or tied). The position-flip gives you confidence that the result isn't an artefact of the question order.

4c — Write prompt-pairwise-001.md

What and why

The findings document captures **the one change you tested** plus the result and your reasoning. Reviewers at DR #1 sometimes go here to ask about your iteration habit — they want to see that you've tested at least one variation, not just shipped the first thing that worked.

Where

docs/prompt-pairwise-001.md .

About to change

Copy the template, fill it in.

Make the change

```
cp <bundle>/AI-RAG_W5_prompt_pairwise_001_template.md docs/prompt-pairwise-001
$EDITOR docs/prompt-pairwise-001.md
```

Fill four sections:

- **What changed** — paste v1 and v2 and *name the one change*.
- **Aggregate result** — paste from the script output.

- **Findings** — read 2-3 v2-win cases and 1-2 v1-win cases; describe what tipped each.
- **Decision** — shipping v2, keeping v1, or iterating? One paragraph with reasoning.

Then commit:

```
git add data/prompt-v1.txt data/prompt-v2.txt \  
      data/pairwise-001-results.json scripts/run_pairwise.py \  
      docs/prompt-pairwise-001.md  
git commit -m "Pairwise 001 — v2 (<one-line change>) wins <N>/20"
```

Run it

```
git log --oneline -1
```

See

```
<sha> Pairwise 001 — v2 (added citation requirement) wins 11/20
```

What happened

You've ran your first prompt comparison and committed the evidence. The iteration pattern from here is: pick one change pairwise it decide. Repeat as needed. This is what's behind every prompt evolution in the rest of the programme.

Step 5 — Critic-Creator on One Hard Question (20 min)

A short exploratory step. You take one harder question from your golden set (g015–g020) and run the Critic-Creator loop on it. The trace shows **how a systematic improvement loop differs from one-shot prompting**.

This isn't a deliverable beyond the trace. It's a pattern recognition exercise — the same loop returns in W17 (reflection), W22 (plan-and-revise), W28 (production self-check).

5a — Pick the question and run the loop

What and why

A hard golden entry is one where your `eval-run-001.md` showed mixed or weak scores. The harder ones (g015–g020) are usually the right candidates because they have room for the loop to improve.

Where

Run `scripts/run_critic_creator.py` from the repo root.

About to change

Nothing in code. Running.

Make the change

Copy the script if needed:

```
cp <bundle>/AI-RAG_W5_run_critic_creator.py scripts/run_critic_creator.py
```

Pick a golden id from your set where eval-run-001 showed an accuracy < 4. (If everything scored 4, pick g019 or g020 — the edge cases often show interesting loop behaviour.)

Run it

```
python scripts/run_critic_creator.py \  
  --golden-id g015 \  
  --creator-model gpt-4o-mini \  
  --judge-model gpt-4o \  
  --max-rounds 3 \  

```

```
--threshold 3.5 \  
--output docs/critic-creator-trace.md
```

See

Per-round logs:

Running Critic-Creator on g015

question : How does the remote work policy interact with the requirement to attend

creator model : gpt-4o-mini

judge model : gpt-4o

max rounds : 3

convergence : mean ≥ 3.5

Round 1: a=3 g=2 f=3 (mean=2.67)

Round 2: a=4 g=3 f=4 (mean=3.67) · CONVERGED

Final answer:

Remote work allowance (up to 3 days per week) is subject to attending mandatory in-p

Trace written to docs/critic-creator-trace.md

Your trace shape will vary. Common patterns:

- **Converges at Round 2.** Most common. Round 1 is broad strokes; Round 2 fixes the gaps.
- **Converges at Round 3.** Hard question; the critic catches multiple issues across rounds.
- **Doesn't converge.** Three rounds without hitting 3.5. Either the threshold is too high, the question is genuinely beyond the creator model's reach, or the critic and creator are arguing — see the reflection step.

What happened

You have a round-by-round trace of an answer improving (or not). The trace is auto-saved as a markdown document, ready for you to add the reflection in 5b.

5b — Read the trace and reflect

What and why

The trace shows *what happened*; the reflection shows *what you make of it*. This is the section reviewers might point at if they're interested in your iteration discipline.

Where

`docs/critic-creator-trace.md` .

About to change

Add a Reflection paragraph at the end of the auto-generated trace.

Make the change

Open `docs/critic-creator-trace.md` . The file has all the rounds rendered plus a `## Reflection` section with a placeholder. Replace the placeholder with 3-5 sentences answering:

- Did the answer materially improve from Round 1 to the final?
- Where did the critic catch real problems (not just rephrase complaints)?
- Anywhere the critic flagged something that wasn't actually wrong (a false alarm)? These are the cases where you'd tighten the rubric.
- Would you trust this loop in production on this question type?

Then commit:

```
git add scripts/run_critic_creator.py docs/critic-creator-trace.md
git commit -m "Critic-Creator trace — g015 (converged at round <N>)"
```

Run it

```
git log --oneline -1
```

See

<sha> Critic-Creator trace — g015 (converged at round 2)

What happened

You have a worked example of the systematic-improvement pattern on one hard question. Step 6 closes M1.

Step 6 — Finalise ADR v1 (Locked) + Prepare DR #1 (45 min)

The M1 push. By the end of this step you have all 6 M1 deliverables committed and a 1-pager ready to defend at DR #1.

6a — Extend ADR 0001 to v1 (Locked)

What and why

Your W1 ADR has been evolving across W1-W4. Today you lock it. “Locked” means: defended at DR #1; no more silent edits; future changes are captured in the change log with reasons.

Where

```
docs/adr/0001-capstone-framing.md .
```

About to change

Replace your existing ADR (or extend it) with the v1 Locked template. The template has 11 sections — most are extensions of what you already have.

Make the change

The bundle template assumes you’ll merge your existing ADR content with the new sections (KPIs, eval baseline, DR #1 defence row, change log).

Two options:

Option A (cleaner): Start from the template and copy your existing ADR text into the right sections.

```
cp <bundle>/AI-RAG_W5_adr_0001_capstone_framing_v1_locked.md \
  docs/adr/0001-capstone-framing.md
$EDITOR docs/adr/0001-capstone-framing.md
```

Option B (more conservative): Open your existing ADR and add the new sections (5 KPIs, 6 Eval baseline, 10 DR #1 defence row, 11 Change log) by hand using the template as a guide.

Whichever you choose, fill these specifically:

- **Section 4 — Decisions locked at M1.** Pull numbers from your Lab 4 comparison and your W5 eval-run-001. Don't leave any cell blank.
- **Section 5 — Sponsor KPIs.** Three of the 8 production KPIs. Pulled from your Stakeholder Map (Lab Step 1). M1 measurement column = today's number; target column = where you want to be by W12 (DR #2).
- **Section 6 — Evaluation baseline.** Paste eval-run-001 aggregate stats.
- **Section 9 — Open questions.** Three things you don't know yet.
- **Section 10 — DR #1 defence.** Leave blank for now — you fill this live during the DR slot.
- **Section 11 — Change log.** Entries for W1 (v0 draft), W4 (cost + model schema-versioning), W5 (locked).

Run it

```
wc -l docs/adr/0001-capstone-framing.md
grep -c "^###" docs/adr/0001-capstone-framing.md
```

See

```
~200 docs/adr/0001-capstone-framing.md
11
```

11 main sections (top-level headings). A filled ADR is typically 150-250 lines.

What happened

M1 deliverable (a) — ADR v1 Locked — is ready. Now the 1-pager.

6b — Write the DR #1 1-pager

What and why

The 1-pager is what your peer reviewers read **before** the DR slot. Aim for one page printed — about 400-500 words. The 1-pager isn't the ADR — it's the *executive summary plus three discussion topics*.

Where

docs/dr/dr1-summary.md .

About to change

Copy the template, fill it in.

Make the change

```
mkdir -p docs/dr
cp <bundle>/AI-RAG_W5_dr1_summary_template.md docs/dr/dr1-summary.md
$EDITOR docs/dr/dr1-summary.md
```

Fill four sections:

- **What you built** — 3-4 sentences. Capstone + user + value story + current state.
- **What you measured** — the eval-run-001 numbers, framed.
- **Top 3 things you want to discuss** — three *specific* questions you want your peers' help on. Resist "any feedback welcome" — peers help most when pointed at a specific question.
- **What you'll defend if asked** — three Q&A bullets you've prepared. Common ones: model choice (from W4 comparison), specific user (from Stakeholder Map), golden set size.

Then also copy the **peer review form** (your peers will fill copies of this for you):

```
cp <bundle>/AI-RAG_W5_dr1_peer_review_form.md docs/dr/dr1-peer-review-form.md
```

Run it

```
wc -w docs/dr/dr1-summary.md
```

See

```
~500 docs/dr/dr1-summary.md
```

If you're over 700 words, trim. The 1-pager is **one page**.

What happened

M1 deliverable (f) — DR #1 1-pager — is ready.

6c — Peer dry-run (15 min)

What and why

Find one cohort peer (even a quick video call). Present your 1-pager + a 60-second /ask demo. Have them ask 3 hard questions. Note what you couldn't answer cleanly — those go into Section 9 (open questions) of your ADR before DR #1.

Where

A call or a shared screen with one peer.

About to change

Possibly your ADR (Section 9) or your 1-pager (Section 5 — Open questions you're sitting with), based on what the dry-run reveals.

Make the change

- Share your `docs/dr/dr1-summary.md` and your repo URL with one peer 30 minutes before the call.
- Present in 5 minutes max. Demo `/ask` live.
- Have the peer ask 3 questions. Note which ones you stumbled on.
- After the call, update your ADR's Open Questions section with the 1-2 questions you couldn't answer cleanly.

Run it

N/A — discussion exercise.

See

A short note in your scratch:

Peer dry-run with <name>:

Q1: <question> — I answered cleanly

Q2: <question> — I stumbled; updated ADR Section 9

Q3: <question> — I answered cleanly but my answer was too long

What happened

You've stress-tested the 1-pager against a real reviewer who isn't the instructor or yourself. The hard questions you couldn't answer cleanly are the ones you'd otherwise get blindsided by in the actual DR. Better to find them now.

6d — Commit M1

What and why

All 6 M1 deliverables in one PR. The PR title is the end-of-Phase-1 marker — it's what shows up in your cohort tracker.

Where

Repo root.

About to change

Final commit + PR.

Make the change

```
# Final sweep — make sure everything is committed
git status

# If you have un-committed work from peer dry-run, commit it
git add docs/adr/0001-capstone-framing.md docs/dr/
git commit -m "M1 — DR #1 1-pager + peer dry-run feedback in ADR"

# Push
git push origin <your-branch>
```

Open a PR titled **“M1 — Phase 1 deliverable”** and tag your two assigned DR #1 peer reviewers.

Run it

```
git log --oneline | head -10
```

See

The last ~8 commits should tell the story of W5:

```
<sha> M1 — DR #1 1-pager + peer dry-run feedback in ADR
<sha> Critic-Creator trace — g015 (converged at round 2)
<sha> Pairwise 001 — v2 (added citation requirement) wins 11/20
<sha> M1 prep — eval-run-001 baseline (avg accuracy 3.45)
<sha> M1 prep — golden_set.jsonl with 20 entries + coverage notes
<sha> M1 prep — stakeholder map locked: HR-policy assistant for...
<sha> W4: tool-calling, real streaming, real cost, schema versioning
...
```

What happened

M1 is committed. All 6 deliverables are in your repo. Your DR #1 slot is scheduled separately — typically Friday evening or Saturday afternoon

the week after W5 weekend. Show up with the 1-pager open and the demo ready.

After M1 — what comes next

Pre-read for W6. Skim the [LangChain Naive RAG tutorial](#) (20 min). Don't copy-paste from it. The point is to see the *shape* — chunk embed retrieve generate. We'll build it ourselves in W6 from scratch.

Your DR #1 slot. Be there 5 minutes early. Have your repo open, your 1-pager open, and a terminal with `uvicorn` already running. The 15 minutes are tight; setup friction kills the first phase.

Phase 2 starts next weekend. W6 — Naive RAG from Scratch. Your golden set is what we measure against from W7 onwards. Your ADR is what evolves. Your `/ask` is what gains a retrieval step inside it. The contract holds.

Phase 1 is yours to defend. Defend it well.